Relatório de Análise Estática do Backend — Katch

Unidade curricular: Laboratório de Desenvolvimento de Software — LDS
Milestone: `m3`
Ficheiro: `m3-relatorio-analise-estatica-backend-v01.pdf`
Pasta de arquivo: `03-artefactos-de-qualidade-de-produto/03.01-backend/03.01.06-analise-estatica`

1. Identificação do documento

Campo	Informação
Grupo	G11
Sistema	Katch
Ano letivo	2026/2027
Curso	LEI — Licenciatura em Engenharia Informática
Turma	LEI3T2
Versão do documento	v01
Issue	`I066` — Configurar a análise estática (SonarQube) e a pipeline CI/CD do backend e documentar a pipeline (Sprint 06)
Executor / Revisor / Auditor	João Borguem / Miguel Santos / Roberto Baptista
Documentos de origem	`m2-convencoes-codigo-quality-gates-v01.md` (`I008`; QG-05), `m3-descricao-pipeline-ci-cd-v01.md`
Versão analisada	`katch-lds/katch-backend`, commit `8d3c91f33a6c3cf8d43d1bd870b4eca6bf79ad46` (ramo da `I066`, PR #40)
Execução da análise	GitHub Actions, workflow `CI`, job `static-analysis`, execução 41737499711, 2026-11-03

1.1. Histórico de versões

Versão	Data	Descrição das alterações	Issue
v01	2026-11-04	Criação do relatório: ferramentas, configuração, regras, âmbito, resultados, problemas relevantes, falsos positivos, ações tomadas e cumprimento do Quality Gate.	`I066`

O relatório é autónomo: a instância SonarQube da pipeline é efémera e não fica acessível depois da execução. Os resultados vêm do artefacto `sonarqube-report` da execução indicada (`resumo.md`, `quality-gate.json` e `problemas.json`).

2. Ferramentas e configuração

Ferramenta	Versão	Onde corre	Configuração
Roslyn Analyzers (.NET)	Do SDK do .NET 10	Em cada build (`dotnet build -warnaserror`, QG-01)	`Directory.Build.props`: `AnalysisLevel latest-recommended`, `EnforceCodeStyleInBuild`, `TreatWarningsAsErrors` e `CodeAnalysisTreatWarningsAsErrors`; `.editorconfig` (convenções de código)
StyleCop.Analyzers	1.2.0-beta.556	Em cada build	`Directory.Build.props` e `stylecop.json` (cabeçalho de copyright, documentação em pt-PT)

Ferramenta	Versão	Onde corre	Configuração
SonarQube Community Build	26.10.0.133078	Job `static-analysis`, em contentor self-hosted criado para cada execução	`scripts/sonarqube/configure.sh` (Quality Gate e Quality Profile) e parâmetros do `dotnet-sonarscanner begin` no `ci.yml`
SonarScanner for .NET	11.3.0	Job `static-analysis`	`.config/dotnet-tools.json`
coverlet e reportgenerator	coverlet.collector 10.1.0; reportgenerator 5.5.11	Jobs `build-and-test` e `static-analysis`	`coverlet.runsettings` (formatos Cobertura e OpenCover)

Os analisadores Roslyn e StyleCop tratam qualquer aviso como erro, por isso o build falha com uma ocorrência de severidade Error e o código integrado não tem nenhuma (QG-05, primeira parte). Os diagnósticos de severidade *suggestion* do `.editorconfig` não bloqueiam o build e são importados pelo SonarQube como problemas externos de severidade Info.

3. Regras

Conjunto	Regras ativas
Quality Profile de C#	«Sonar way core», o perfil base da Sonar para C# na Community Build 26.10, que corresponde ao «Sonar way» das convenções de código (secção 5). Nesta versão não existe um perfil com o nome exato «Sonar way»; o script de configuração usá-lo-ia, se existisse.
Quality Profiles de JSON e YAML	«Sonar way comprehensive» (ficheiros de configuração do repositório)
Roslyn Analyzers	Conjunto `latest-recommended`
StyleCop.Analyzers	Regras predefinidas, com as definições de `stylecop.json`

3.1. Quality Gate «Katch QG-05»

As condições aplicam-se ao código total, porque a instância é nova em cada execução e, numa primeira análise, as condições de código novo do «Sonar way» não se aplicariam. As classificações são as do modo MQR (Multi-Quality Rule).

Métrica	Falha quando	Valor obtido	Estado
Fiabilidade (`software_quality_reliability_rating`)	Pior do que B	A	OK
Segurança (`software_quality_security_rating`)	Pior do que A	A	OK
Manutenção (`software_quality_maintainability_rating`)	Pior do que A	A (dívida técnica de 0,0%)	OK
Duplicação (`duplicated_lines_density`)	Acima de 3%	0,3%	OK

4. Âmbito

Aspeto	Âmbito
Código analisado	Todo o repositório `katch-backend`: `src/Katch.Backend` (código principal), `tests/Katch.Backend.UnitTests` e `tests/Katch.Backend.IntegrationTests` (código de teste) e ficheiros de configuração JSON e YAML. Sem exclusões de análise (convenções de código, secção 5).

Aspeto	Âmbito
Ficheiros	393 ficheiros indexados; 3 projetos MSBuild (1 principal e 2 de testes); 39 ficheiros ignorados pelo `.gitignore`
Linhas de código	10 036
Código gerado	As migrations do Entity Framework têm o cabeçalho `<auto-generated>` e são tratadas pelo SonarQube como código gerado
Cobertura	Importada dos relatórios OpenCover dos testes unitários e de integração. Excluídos da medida, como no QG-04: `Infrastructure` (com as migrations), `BackgroundJobs`, `Dtos` e `Program.cs`

5. Resultados

Indicador	Valor
Estado do Quality Gate	**Aprovado (OK)**
Fiabilidade	A
Segurança	A
Manutenção	A (dívida técnica 0,0%)
Duplicação	0,3%
Cobertura de linhas (âmbito do QG-04)	99,7% no SonarQube; 99,6% no reportgenerator (QG-04, mínimo de 70%)
Security hotspots	0
Problemas em aberto	162, todos de severidade Info (secção 6.3)

O build com os analisadores Roslyn e StyleCop como erro terminou sem avisos na mesma execução (job `build-and-test`, passo «QG-01 Build»).

6. Problemas

6.1. Primeira análise

A primeira análise completa (2026-11-03, antes das correções) encontrou 177 problemas e o Quality Gate falhou com fiabilidade C e segurança C. Dos 177 problemas, 15 eram do analisador SonarQube ou do xUnit e 162 eram sugestões de estilo do Roslyn.

Regra	N.º	Impacto	Onde	Decisão
S2699 — teste sem asserção	1	Manutenção, bloqueante (código de teste)	`LocationTests.CoordinateLimitsAreAccepted`	Corrigido (6.2)
xUnit2032 — `Assert.IsAssignableFrom`	3	Manutenção, Info (código de teste)	`ExceptionFiltersTests`, `JobExplorationExceptionFilterTests`, `CandidateQuotaTests`	Corrigido (6.2)
S6964 — propriedade de tipo valor num pedido	6	Fiabilidade, médio	`JobRequest`, `UpdateCandidateProfileRequest`, `CompanyRegistrationRequest`, `CreateRecruiterAccountRequest`, `RegisterCandidateRequest` (2)	Falso positivo (6.4)
S6932 — ler o formulário em vez de `IFormFile`	3	Fiabilidade, segurança e manutenção, médio	`ReadUploadAsync` de `CandidateProfileController`, `CompanyController` e `JobsController`	Decisão de desenho aceite (6.4)

Regra	N.º	Impacto	Onde	Decisão
S2068 — credencial no código	2	Segurança, médio	`FieldReasons.PasswordPolicy` e `FieldReasons.PasswordTooLong`	Falso positivo (6.4)
IDE0007, IDE0008, IDE0028 — estilo	162	Manutenção, Info	Testes (152) e código principal (10)	Mantidos como sugestões (6.3)

6.2. Problemas corrigidos

Regra	Correção	Commit
S2699	O teste `CoordinateLimitsAreAccepted` passa a verificar a latitude e a longitude dos dois limites, em vez de só criar as localidades.	`8d3c91f`
xUnit2032	`Assert.IsAssignableFrom<T>(…)` passa a `Assert.IsType<T>(…, exactMatch: false)`, com o mesmo significado e o nome que a regra recomenda.	`8d3c91f`

6.3. Problemas mantidos (severidade Info)

Os 162 problemas em aberto são sugestões do `.editorconfig` importadas do build: IDE0007 (usar `var`, 100), IDE0008 (usar o tipo explícito, 52) e IDE0028 (simplificar a inicialização de coleções, 10). Correspondem às preferências `csharp_style_var_*` com severidade *suggestion* das convenções de código, que não são erros. Não afetam nenhuma classificação nem o Quality Gate, e 152 estão em código de teste. Ficam como proposta de melhoria: uniformizar o uso de `var` e das expressões de coleção numa Issue própria.

6.4. Falsos positivos e decisões de desenho

A instância é efémera e não permite marcar problemas no painel. Os falsos positivos são registados no `ci.yml`, em `sonar.issue.ignore.multicriteria`, com uma entrada por regra e por ficheiro (sem exclusões genéricas), e ficam visíveis na revisão do código.

Regra	Ficheiros	Justificação
S6964	`JobRequest.cs`, `UpdateCandidateProfileRequest.cs`, `CompanyRegistrationRequest.cs`, `CreateRecruiterAccountRequest.cs`, `RegisterCandidateRequest.cs`	A regra avisa que um campo de tipo valor em falta no pedido fica com o valor por omissão sem ninguém saber (*under-posting*). Aqui esse caso está tratado: `LocationId` igual a `Guid.Empty` devolve 400 ERR-10 com «O campo é obrigatório.», e `acceptTerms` em falta devolve o ERR-10 do RF003 e do RF037. As alternativas sugeridas (`required` ou `[JsonRequired]`) trocariam esta resposta pelo erro genérico de desserialização, fora do contrato da API v02 (secção 4).
S6932	`CandidateProfileController.cs`, `CompanyController.cs`, `JobsController.cs`	O `UploadRequestFilterAttribute` verifica o tipo de conteúdo e a dimensão do pedido antes do model binding, para responder ERR-13 (415) e ERR-12 (413) como exige a API v02 (secção 2.6). Por isso o ficheiro é lido do formulário depois do filtro, e não por binding de `IFormFile`. O conteúdo é sempre validado pelo `FileValidator` (assinatura, extensão e dimensão; RNF016) antes de ser guardado.
S2068	`FieldReasons.cs`	As duas ocorrências são mensagens de erro para o utilizador («A palavra-passe tem de ter…»), que a regra assinala por conterem a palavra «palavra-passe». Não há credenciais no código.

6.5. Problemas relevantes

Na versão analisada não há problemas de fiabilidade, de segurança ou de manutenção acima de Info em aberto, nem security hotspots. Os problemas relevantes encontrados na primeira análise foram corrigidos (6.2) ou justificados (6.4).

7. Cumprimento do Quality Gate (QG-05)

Parte do QG-05	Resultado
Sem ocorrências de severidade Error nos analisadores Roslyn e StyleCop	Cumprido: build com `-warnaserror` sem avisos (job `build-and-test`)
Quality Gate do SonarQube aprovado	Cumprido: «Katch QG-05» com estado OK nas quatro condições (secção 3.1)

O bloqueio pelo Quality Gate foi demonstrado na execução 41740891672 (PR #41): uma cadeia de ligação com `Password=` e um valor fictício fez a segurança passar a pior do que A e o job `static-analysis` falhar, com o merge bloqueado (relatório de evidências da pipeline, E5).

8. Reprodução

1. Localmente: `docker compose -f scripts/sonarqube/docker-compose.yml up -d`, `SONAR_ADMIN_PASSWORD=<palavra-passe> bash scripts/sonarqube/configure.sh` e a análise com os mesmos parâmetros do passo «QG-05 Iniciar a análise» do `ci.yml`, usando o token de `.sonar-token`. O painel fica em `http://localhost:9000`.
2. Na pipeline: qualquer Merge Request para a `main`, ou *Run workflow* no workflow `CI`. O resultado fica no resumo da execução e no artefacto `sonarqube-report`.